

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №I по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Баранова Д.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 08.10.26

Москва, 2026

Постановка задачи

Вариант 2.

Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; – создает дочерний процесс.
- `int pipe(int *fd)`; – создает неименованный канал.
- `ssize_t read(int fd, void *buf, size_t count)` — чтение данных из файлового дескриптора.
- `ssize_t write(int fd, const void *buf, size_t count)` — запись данных в файловый дескриптор.
- `int dup2(int oldfd, int newfd)` — дублирование файлового дескриптора.
- `int open(const char *pathname, int flags, mode_t mode)` — открытие/создание файла. Флаги `O_WRONLY` | `O_CREAT` | `O_TRUNC` обеспечивают запись, создание при отсутствии и очистку при существовании.
- `int close(int fd)` — закрытие файлового дескриптора.
- `int execlp(const char *file, const char *arg, ...)` — замена образа текущего процесса новой программой.
- `pid_t wait(int *status)` — ожидание завершения дочернего процесса.

Алгоритм `parent.cpp`:

Считать имя файла с терминала через `read` (посимвольно, до `\n` или EOF).

Проверить ввод: -1 — ошибка, 0 — пусто. При ошибке — выход.

Создать канал `pipe(fd)`.

Создать дочерний процесс `fork()`.

В ребёнке: закрыть `fd[1]`, через `dup2(fd[0], STDIN_FILENO)` перенаправить `stdin` на канал, закрыть `fd[0]`, запустить `execlp("./child", "child", filename, NULL)`.

В родителе: закрыть `fd[0]`, в цикле считывать строки с числами из `stdin` и писать в `fd[1]`. По EOF закрыть `fd[1]` (сигнал EOF для ребёнка) и вызвать `wait(NULL)`.

Алгоритм `child.cpp`:

Проверить `argc < 2` — если нет имени файла, выйти с ошибкой.

Открыть файл через `open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644)`.

В цикле читать строки из stdin (это канал, перенаправленный родителем).

Каждую строку разбить на токены через strtok, каждый токен преобразовать в double через strtod, сложить.

Сумму преобразовать в строку через std::to_string, добавить \n, записать в файл через write.

По EOF закрыть файл. При ошибке чтения — выйти с сообщением.

Код программы

parent.cpp

```
#include <unistd.h>
#include <sys/wait.h>
#include <cstdlib>
#include <fcntl.h>
#include <sys/types.h>
```

```
#define MAX_LINE 1024
```

```
ssize_t read_line(int fd, char* buf, size_t max){
    size_t i = 0;
    char c;
    while (i < max - 1){
        ssize_t n = read(fd, &c, 1);
        if (n == 0) break;
        if (n < 0) return -1;
        buf[i++] = c;
        if (c == '\n') break;
    }
    buf[i] = '\0';
    return i;
}
```

```
int main(){
    char filename[MAX_LINE];
    write(STDOUT_FILENO, "Enter filename: ", 16);
    ssize_t n = read_line(0, filename, MAX_LINE);
    if (n == -1){
        write(STDERR_FILENO, "read error\n", 11);
        std::exit(EXIT_FAILURE);
    }
    if (n == 0){
        write(STDERR_FILENO, "no filename\n", 12);
        std::exit(EXIT_FAILURE);
    }
}
```

```

    }
    if (filename[n - 1] == '\n'){
        filename[n - 1] = '\0';
        n--;
    }
    if (n == 0){
        write(STDERR_FILENO, "empty filename\n", 15);
        std::exit(EXIT_FAILURE);
    }

    int fd[2];
    if (pipe(fd) < 0){
        write(STDERR_FILENO, "pipe error\n", 11);
        std::exit(EXIT_FAILURE);
    }

    pid_t pid = fork();
    if (pid == -1){
        write(STDERR_FILENO, "fork error\n", 11);
        std::exit(EXIT_FAILURE);
    } else if (pid == 0){
        close(fd[1]);
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        execlp("./child", "child", filename, NULL);
        write(STDERR_FILENO, "exec error\n", 11);
        std::exit(EXIT_FAILURE);
    } else{
        close(fd[0]);
        char buffer[MAX_LINE];
        write(STDOUT_FILENO, "Enter numbers: ", 15);
        ssize_t len;
        while((len = read_line(STDIN_FILENO, buffer, MAX_LINE)) > 0){
            write(fd[1], buffer, len);
        }
        close(fd[1]);
        wait(NULL);
    }
}

```

child.cpp

```

#include <unistd.h>
#include <cstdlib>
#include <cstring>
#include <fcntl.h>
#include <string>

#define MAX_LINE 1024

ssize_t read_line(int fd, char* buf, size_t max){
    size_t i = 0;
    char c;
    while (i < max - 1){
        ssize_t n = read(fd, &c, 1);
        if (n == 0) break;
        if (n < 0) return -1;
        buf[i++] = c;
        if (c == '\n') break;
    }
    buf[i] = '\0';
    return i;
}

int main(int argc, char **argv){
    if (argc < 2){
        write(STDERR_FILENO, "no filename\n", 12);
        std::exit(EXIT_FAILURE);
    }
    int file_fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (file_fd == -1){
        write(STDERR_FILENO, "open failed\n", 12);
        std::exit(EXIT_FAILURE);
    }
    ssize_t len;
    char buffer[MAX_LINE];
    while((len = read_line(STDIN_FILENO, buffer, MAX_LINE)) > 0){
        char* token = strtok(buffer, " \t\n");
        double sum = 0;
        while(token != NULL){
            double value = strtod(token, NULL);
            sum += value;
            token = strtok(NULL, " \t\n");
        }
    }
}

```

```

    }
    std::string s = std::to_string(sum)+"\n";
    write(file_fd, s.c_str(), s.length());
}
if (len == -1){
    write(STDERR_FILENO, "read error\n", 11);
    std::exit(EXIT_FAILURE);
}
close(file_fd);
return 0;
}

```

Протокол работы программы

diana@DESKTOP-NSHR21B:~/lab1\$./parent

Enter filename: test

Enter numbers: 1 9.78 34.9001\

45.7 0.0002

diana@DESKTOP-NSHR21B:~/lab1\$ cat test

45.680100

45.700200

diana@DESKTOP-NSHR21B:~/lab1\$./parent

Enter filename:

empty filename

diana@DESKTOP-NSHR21B:~/lab1\$./parent

Enter filename: test

Enter numbers: 5.0 3.22 abc 5

bad number: abc, skipped

diana@DESKTOP-NSHR21B:~/lab1\$ cat test

13.220000

Вывод

В ходе работы я научилась работать с системными вызовами и процессами.

Программа выполняет поставленную задачу, корректно передает данные между процессами и проходит тесты.